From-Scratch Decision Trees and SVMs for Azerbaijani Property Listings

Bakir Gahramanov and the ML Final Project team

MLE-AI-201 Machine Learning, AI Academy — Cohort II

***Abstract*—**We study sale-price regression and premium-price classification on a scraped Azerbaijani property-listings dataset. The team implemented decision trees for both tasks and a Pegasos support-vector machine in NumPy, including a random Fourier feature extension. After deduplication, 64,454 listings remained. A seeded 70/15/15 split, training-only preprocessing, validation-based configuration selection, and three-fold training cross-validation support a held-out comparison with scikit-learn. Our regression tree reached 103,348 AZN MAE and 0.227 R^2 on the test set, close to the matched baseline. The tree classifier reached 0.904 F1; linear and nonlinear Pegasos reached 0.777 and 0.806 F1. Very expensive listings dominated price RMSE: the 85 test listings above the training 99th-percentile price had 3.64 million AZN mean absolute error. The results show that the from-scratch models can closely match reference trees, while the heavy price tail remains a practical limitation. Code: `github.com/BulqeyisMammadova/ML_Final_Project`.

***Index Terms*—**decision tree, support-vector machine, Pegasos, regression, classification, real-estate price prediction.

I. Introduction

Real-estate asking prices vary with area, location, floor and property type. A scraped listing dataset adds duplicate advertisements, missing values and extreme prices, so the modelling problem includes careful data preparation as well as algorithm implementation. We frame the same listings as two tasks: predict sale price and classify whether a listing is above the training-set median price.

Our team built decision trees and Pegasos SVMs from scratch in NumPy, then compared them with scikit-learn models under a shared split and matched settings where possible. We report validation curves, held-out metrics, training-only cross-validation, runtime and error analysis. The objective is a reproducible comparison that explains model behaviour, including where high-price listings challenge regression.

II. Related Work / Background

Decision trees choose feature thresholds that reduce node impurity and recursively partition the feature space. Gini and entropy serve the classification task; variance reduction serves price regression. Their interpretable splits can model nonlinear interactions but can overfit without depth and leaf-size controls. A soft-margin SVM instead optimises regularised hinge loss. Pegasos uses stochastic sub-gradients with a decreasing learning rate [1]. Random Fourier features provide a finite-dimensional approximation to an RBF kernel and allow the same linear solver to represent nonlinear boundaries.

III. Data and Preprocessing

The Kaggle bina.az sale dataset [2] has 100,775 rows before cleaning. We keep the latest scrape of each listing URL, leaving 64,454 unique listings. The price currency in the downloaded version is AZN. Area strings are parsed into square metres, including conversion from *sot*; floor and total-floor counts are extracted from combined strings. The model uses area, floor, total floors, coordinates, room count, land area and one-hot property type (14 encoded features). We remove `unit_price` and `total_price` because they are calculated from the target, and we exclude listing identifiers, names and raw addresses. Missing numeric values use training medians; unavailable land area uses zero because it is inapplicable to many flats.

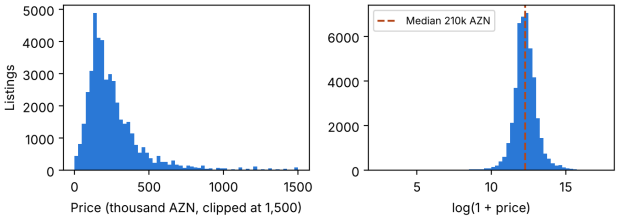


Figure 1. Training-set sale prices (left, clipped at 1.5 million AZN for readability) and $\log(1+\text{price})$ with the 210,000 AZN premium threshold (right).

The target is positive sale price; regression fits $\log(1+\text{price})$ and returns predictions to AZN for evaluation. The binary label is premium when price exceeds the training median of 210,000 AZN. The split sizes are shown in Table 1; the validation and test halves are stratified by this fixed label. Exploratory analysis found a heavy right tail and some suspicious low prices or possible rental entries. An exploratory 99th-percentile training filter reduced typical MAE but increased RMSE on unchanged validation data, so the integrated pipeline retains high-price observations rather than applying an unsupported universal outlier cutoff.

Figure 1 shows the training-price distribution. Raw prices are strongly right-skewed (the mean is 1.57 times the median), while $\log(1+\text{price})$ is close to symmetric, which motivates fitting the regression models on the log scale. The median split gives an almost balanced tier label (49.5 % premium in training).

Table 1. Dataset summary after cleaning.

Property	Value
Rows (after cleaning)	64,454
Encoded features used	14
Train / Val / Test split	45,118 / 9,667 / 9,669
Premium share (train)	49.5%

IV. Method

A. Decision tree (from scratch)

For each numeric feature, the tree sorts training values, considers midpoints between distinct adjacent values, and selects the threshold with the greatest weighted impurity decrease. Classification supports Gini impurity and entropy; regression uses mean squared deviation within nodes. Recursion stops according to maximum depth, minimum split and leaf sample counts, or minimum impurity decrease. A regression leaf predicts its mean target, while a classification leaf predicts its majority class and exposes class-frequency probabilities. All split search, recursion and prediction are implemented in NumPy. Tree depth, leaf size and classification criterion are selected on validation data.

To keep the split search fast, each feature is sorted once and the left/right class counts (or $\sum y$ and $\sum y^2$) for every candidate threshold are obtained with cumulative sums; this returns the same split as the naive search and was about $180\times$ faster on 3,000 rows. A unit test builds our tree and scikit-learn's on small tie-free examples (Gini, entropy and MSE) and checks that every split feature, threshold and leaf matches. On the real data we found that the midpoint $(a+b)/2$ of two almost equal coordinates can round to b in floating point, sending every sample to one child and causing infinite recursion; as in scikit-learn we fall back to a in that case and refuse empty splits, and a regression test guards it.

B. Random forest from our own trees (bonus)

For Problem 9 we averaged 50 of our own trees. Each tree is grown on a bootstrap sample of the training rows and considers only two randomly chosen features at each split (`max_features`$=2$, `min_samples_leaf`$=5$); features that are constant in a node are skipped, as in scikit-learn. Regression averages the tree predictions; classification averages the leaf class frequencies.

C. Support-vector machine (from scratch)

We map the standard and premium labels to -1 and $+1$, respectively, and minimize the regularized hinge-loss objective:

$$J(\mathbf{w},b) = \tfrac{\lambda}{2}\|\mathbf{w}\|^2 + \tfrac{1}{n}\sum_{i=1}^{n}\max\bigl(0,\,1-y_i(\mathbf{w}^\top\mathbf{x}_i+b)\bigr). \quad (1)$$

For the linear model, the margin width is $2/\|\mathbf{w}\|_2$. We identify training points on or inside the margin as support vectors:

$$y_i(\mathbf{w}^\top\mathbf{x}_i+b) \le 1. \quad (2)$$

At update t, Pegasos samples training example i_t and uses $\eta_t = 1/(\lambda t)$. Its stochastic updates are

$$\mathbf{w}_t = (1-\eta_t\lambda)\mathbf{w}_{t-1} + \eta_t y_{i_t}\mathbf{x}_{i_t}\mathbb{1}\Big[y_{i_t}(\mathbf{w}_{t-1}^\top\mathbf{x}_{i_t}+b_{t-1})<1\Big], \quad (3)$$

$$b_t = b_{t-1} + \eta_t y_{i_t}\mathbb{1}\Big[y_{i_t}(\mathbf{w}_{t-1}^\top\mathbf{x}_{i_t}+b_{t-1})<1\Big]. \quad (4)$$

The bias is unregularized: it is updated on margin violations but is not multiplied by the weight-decay factor. For the nonlinear model, random Fourier features approximate an RBF kernel:

$$\begin{aligned}\mathbf{z}(\mathbf{x}) &= [z_j(\mathbf{x})]_{j=1}^{D},\\ z_j(\mathbf{x}) &= \sqrt{\tfrac{2}{D}}\cos(\phi_j(\mathbf{x})),\\ \phi_j(\mathbf{x}) &= \boldsymbol{\omega}_j^\top\mathbf{x}+c_j,\\ \boldsymbol{\omega}_j &\sim \mathcal{N}(0,2\gamma\mathbf{I}),\\ c_j &\sim \mathcal{U}(0,2\pi), \quad j=1,\dots,D.\end{aligned} \quad (5)$$

Pegasos is applied to $\mathbf{z}(\mathbf{x})$; its transformed-space coefficients are not interpreted as original-feature importance. We chose random Fourier features rather than kernelised Pegasos because a kernel model must store a coefficient per training example and evaluate kernels against the whole training set at prediction time, which is costly for 45,118 rows. With RFF the cost is fixed by D (256 components), the same linear Pegasos code is reused unchanged, and the bandwidth γ remains a single tunable hyperparameter.

V. Experimental Setup

All experiments used seed 42. After deduplication, 70% of the rows were reserved for training. The premium threshold was set to the training price median; the remaining rows were split into validation and test halves stratified by that label. Missing numeric features were filled with training medians, except inapplicable land area, which used zero. Property types were one-hot encoded using training categories only. The team's SVM used training means and standard deviations; scikit-learn's hinge-loss baseline scaled features inside a pipeline. Every cross-validation fold refitted its own preprocessing on that fold's training rows.

Tree depth, minimum leaf size, Gini versus entropy, SVM regularisation, and RFF bandwidth were chosen from validation results only. The regression selection metric was log-RMSE; classification used premium-class F1. The test rows were evaluated once with the selected settings. We report price-scale RMSE, MAE, and R^2, plus precision, recall, F1, ROC-AUC, and confusion matrices for price tiers. NumPy implementations computed the metrics. We compared the trees with matched scikit-learn trees and the linear Pegasos SVM with a scaled SGD hinge-loss classifier. Wall-clock fit and prediction times were measured separately and recorded by the reproduction script; they depend on the machine.

The headline numbers come from `python -m src.run_all`. The additional overfitting curve, EDA figure and SVM schedule study use the same split and training/validation rows only (`python -m src.report_extras`). The decision-tree experiments and the random forest bonus (`python -m src.exp_tree`) and the geospatial bonus (`python geospatial_bonus.py`)

Table 2. Task A: held-out price regression. RMSE and MAE are in thousands of AZN. CV log-RMSE is training-fold mean ± standard deviation.

Model	RMSE	MAE	R^2	CV log-RMSE
Our tree	759.1	103.3	0.227	0.409 ± 0.018
sklearn tree	759.2	103.5	0.227	0.408 ± 0.018

Table 3. Task B: held-out premium-class classification. CV F1 is training-fold mean ± standard deviation.

Model	Test F1	Test AUC	CV F1
Our tree	0.904	0.967	0.897 ± 0.001
sklearn tree	0.904	0.967	0.897 ± 0.001
Our linear Pegasos	0.777	0.826	0.774 ± 0.013
sklearn SGD hinge	0.778	0.839	0.778 ± 0.000
Our RFF Pegasos	0.806	0.874	0.807 ± 0.006

were developed earlier on the shared `get_splits()` split with 9,668 validation and 9,668 test rows; their numbers are therefore reported separately and compared only with models trained on the same rows.

VI. Results

The reproducible run retained 64,454 unique listings. The seeded split contained 45,118 training, 9,667 validation, and 9,669 test rows. The premium threshold was the training median of 210,000 AZN. The 14 encoded features included area, floors, coordinates, room count, land area, and one-hot property type. The category vocabulary came from training rows only. All configurations below were selected using validation results before the test set was evaluated. Three-fold cross-validation (CV) uses training rows only and is reported separately from the single held-out test score.

Validation selected depth 12 for regression, depth 10 for classification, and at least 20 samples per leaf for both trees; the classifier used entropy. The linear SVM used $\lambda = 10^{-3}$ and 20,000 updates. The random Fourier feature (RFF) SVM used $\gamma = 1$, 256 components, $\lambda = 10^{-3}$, and 12,000 updates. Figure 2 shows how tree depth, minimum leaf size, and the classification split criterion affected validation scores. The two tree implementations were very close on both tasks (Tables 2 and 3); small differences can arise from threshold arithmetic and tie-breaking.

Figure 3 shows the bias–variance story for the regression tree with no leaf-size limit. Training log-RMSE keeps falling with depth (0.68 at depth 2, 0.09 at depth 20), while validation log-RMSE is lowest at depth 10 (0.430) and then rises to 0.461: deeper trees memorise the training rows. Requiring at least 20 samples per leaf reduces this variance, and the selected depth-12, leaf-20 tree reaches validation log-RMSE 0.416.

The linear Pegasos model learned $\|w\| = 3.464$ and a margin width of $2/\|w\| = 0.577$ in standardised feature space. By the criterion $y_i(w^\top x_i + b) \le 1$, 23,410 of 45,118 training rows were support vectors. Its regularised hinge objective fell from 15.091 after 100 updates to 0.580 after 20,000 updates, with stochastic fluctuations (Fig. 4). The RFF model improved the held-out F1 over linear Pegasos from 0.777 to

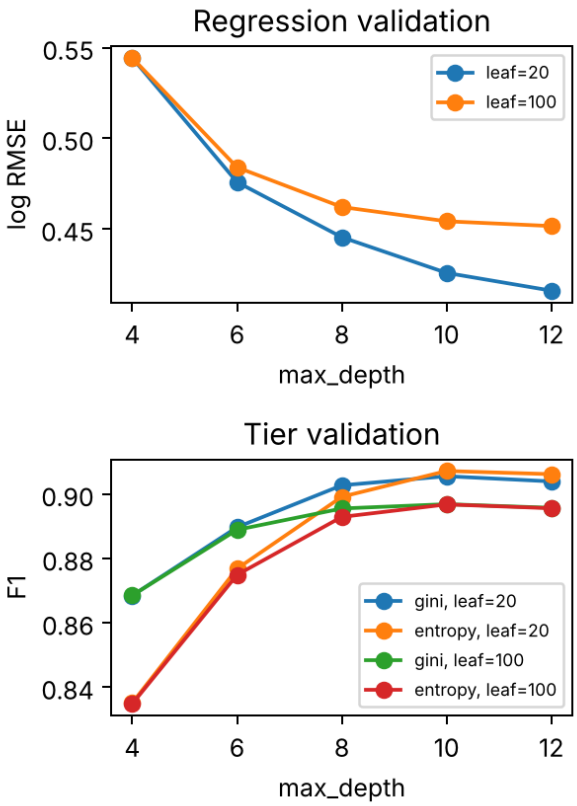


Figure 2. Decision-tree validation curves used for configuration selection. Lower is better for regression log-RMSE; higher is better for classification F1.

Table 4. Linear Pegasos: effect of λ (20,000 updates, validation set).

λ	Margin $2/\|\mathbf{w}\|$	Val F1
10^{-2}	0.860	0.771
10^{-3} (selected)	0.577	0.772
10^{-4}	0.087	0.690

0.806; its three-fold CV F1 was 0.807 ± 0.006. On validation, $\gamma = 1$ beat $\gamma = 0.1$ (F1 0.796 vs. 0.785 at $\lambda = 10^{-3}$).

Table 4 shows how regularisation trades margin for fit. A larger λ shrinks $\|\mathbf{w}\|$ and widens the margin (0.86 at $\lambda = 10^{-2}$); a very small λ gives a narrow margin (0.09) and the lowest validation F1, because with the $1/(\lambda t)$ step the early updates are huge and 20,000 updates are not enough to settle. Table 5 varies the number of updates and the learning-rate schedule at $\lambda = 10^{-3}$. With the Pegasos step $1/(\lambda t)$, more updates lower the objective, widen the margin and raise validation F1. The slower-decaying $1/(\lambda\sqrt{t})$ step keeps taking large steps, so $\|\mathbf{w}\|$ stays large (margin below 0.05), the objective stays high and F1 is worse; we therefore kept the Pegasos schedule.

The tree classifier had 470 false positives and 441 false negatives on the held-out set. Area contributed 54.3% of its total impurity decrease, followed by longitude (19.2%) and latitude (9.6%). For the regression tree, area contributed 59.0%, latitude and longitude 11.8% and 11.6%, and the land

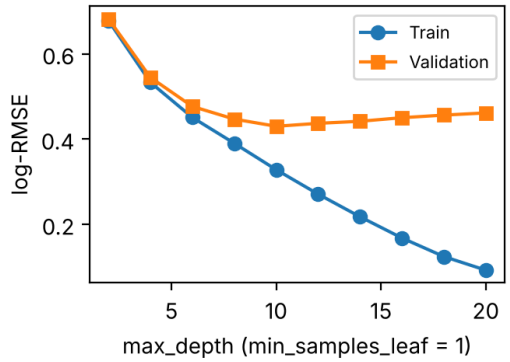


Figure 3. Overfitting of our regression tree: training and validation log-RMSE as `max_depth` grows (`min_samples_leaf`= 1). The widening gap after depth 10 is overfitting.

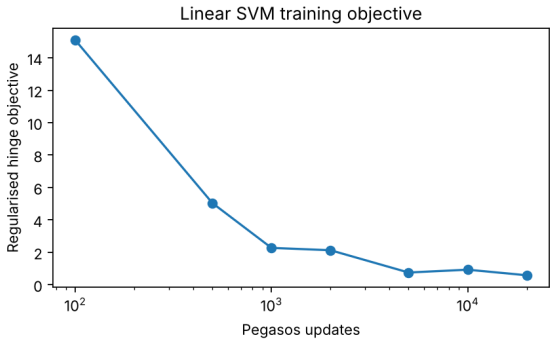


Figure 4. Training objective $J(\mathbf{w}, b)$ of the linear Pegasos SVM ($\lambda = 10^{-3}$) as updates are added.

property type and land area about 7% each. These are predictive associations, not causal effects.

Because the linear SVM is trained on standardised features, its weights are comparable in size. The largest positive weights are room count (2.28), the office type (1.25) and area (1.09); the garage type has the most negative weight (-1.65). Room count and area are strongly correlated, so the SVM splits their shared effect differently from the tree, which puts most importance on area. A linear boundary cannot represent the location effect as well as the tree, which partly explains the gap between linear Pegasos (F1 0.777) and the tree (F1 0.904).

Table 6 lists fit and prediction times. scikit-learn's compiled trees fit 3–5× faster than our NumPy trees; our linear Pegasos fit is as fast as `SGDClassifier`. The RFF model is slower to predict because every input is mapped to 256 cosine features.

Price errors were much larger in the luxury tail. The 85 test listings above the training 99th-percentile price of 2.5 million AZN had mean absolute error 3.64 million AZN, versus 72.0 thousand AZN for the remaining 9,584 test listings. The median absolute error across the whole test set was 24.7 thousand AZN. Our tree underpredicted 90.6% of those 85 tail listings, indicating systematic compression of high prices. The reproducible run also saves a log-scale actual-versus-predicted plot for detailed inspection.

Table 5. Linear Pegasos: number of updates and learning-rate schedule ($\lambda = 10^{-3}$, validation set).

Step η_t	Updates	Objective	Margin	Val F1
$1/(\lambda t)$	2,000	2.12	0.110	0.692
$1/(\lambda t)$	5,000	0.75	0.269	0.727
$1/(\lambda t)$	20,000	0.58	0.577	0.772
$1/(\lambda \sqrt{t})$	2,000	43.94	0.016	0.652
$1/(\lambda \sqrt{t})$	5,000	15.08	0.025	0.614
$1/(\lambda \sqrt{t})$	20,000	12.20	0.048	0.604

Table 6. Fit and prediction time in seconds on the full training and test sets (one run, machine-dependent).

Model	Fit (s)	Predict (s)
Our tree, regression	0.66	0.033
sklearn tree, regression	0.19	0.002
Our tree, classification	0.89	0.037
sklearn tree, classification	0.17	0.003
Our linear Pegasos	0.11	$<$0.001
sklearn SGD hinge	0.18	0.003
Our RFF Pegasos	0.48	0.178

Random forest (bonus). This experiment was run with the tree experiment script on its own seeded split (Task A on log price), so we compare it only with a single tree on the same rows. On the held-out rows the forest raised R^2 from 0.781 to 0.832 (log-MAE 0.202 to 0.170) on Task A and F1 from 0.908 to 0.934 (ROC-AUC 0.938 to 0.984) on Task B; scikit-learn's `RandomForestRegressor` with the same settings reached $R^2 = 0.810$. Validation R^2 grows quickly with the number of trees and flattens after about 30 (Fig. 5).

Geospatial clusters (bonus). For Problem 10 we converted the coordinates of Baku listings to kilometres around the training median and fitted k-means on training coordinates only (Fig. 6); validation and test listings were assigned to the learned centres. Validation silhouette scores were 0.667 for $k = 2$ and 0.581 for $k = 5$; we show $k = 5$ because it gives more geographic detail, not because it scored best. Median training prices differ strongly between clusters, from 88,000 AZN in the north-western cluster to 250,000–265,000 AZN in the central and eastern clusters. This supports the large tree importance of latitude and longitude: location carries price information that the linear SVM can only use as a straight-line trend. The analysis is descriptive and does not change the models above.

VII. Discussion and Limitations

Validation curves show that allowing deeper trees improves both tasks over the shallowest settings, but without a leaf-size limit validation error rises again after depth 10 while training error keeps falling; minimum leaf size constrains this variance, and averaging bootstrapped trees in the forest reduces it further. Matching scikit-learn trees give nearly identical test scores, while their training is faster in this environment. Our SVM's learned margin and performance depend on feature scaling, regularisation and the stochastic update budget. RFF improves premium-class F1, but its transformed coefficients cannot be read as original-feature

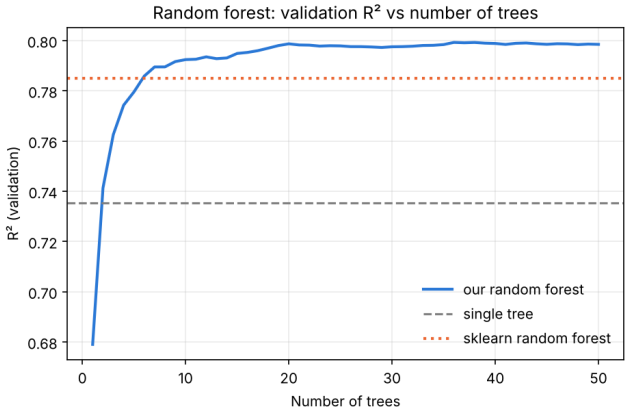


Figure 5. Bonus: validation R^2 (log price) of our random forest as trees are added, compared with a single tree and scikit-learn's forest.

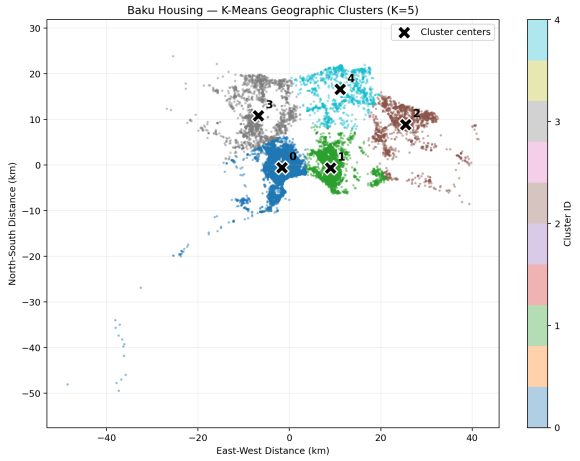


Figure 6. Bonus: k-means ($k = 5$) clusters of Baku listings fitted on training coordinates.

effects.

The price distribution has an extreme upper tail and possible mislabeled sale/rental listings. The model compresses expensive predictions, so its overall RMSE is much larger than its median error. Property-type and coordinate features only approximate neighbourhood and condition; unavailable text, sparse categories and imperfect listing labels limit generalisation. Cross-validation standard deviations describe variation across three training folds, not a confidence interval for the held-out test result. Timings are machine-specific and the SVM and scikit-learn optimisers do not have identical update rules.

VIII. Conclusion

The from-scratch trees closely matched scikit-learn on both sale-price and price-tier tasks. The RFF Pegasos model improved tier F1 over its linear version, while luxury properties remained the main regression failure. As bonuses, a random forest built from our own trees improved both tasks over a single tree, and k-means clusters of the coordinates showed clear location-based price segments. Future work should verify suspicious listings, improve location and property-condition features, and evaluate models on a later scrape to measure performance under market change.

Team Contributions

Member 1 (nhesen): data loading and cleaning (`data_prep.py`), EDA, data-quality notes and the geospatial bonus. *Member 2 (Bulqeyis Mammadova):* the from-scratch decision tree, its sklearn-matching tests, the tree experiments and figures, and the random forest bonus. *Member 3 (Ulvi):* the from-scratch Pegasos SVM and its RFF extension, the SVM experiments and figures. *Member 4 (Bakir Gahramanov):* the integrated reproduction runner, metrics, scikit-learn baselines, integration tests, README and report integration. Git history in the repository records each contribution.

Tools and Acknowledgements

An AI coding assistant helped draft and test the integration runner, evaluation metrics, baseline comparisons, README, and this Results section. For the decision tree and the random forest, an AI assistant was used as a tutor: it explained the concepts and proposed code step by step; the team member typed, ran, tested and debugged the code and can explain it. An AI assistant also helped draft the additional report analyses (overfitting curve, SVM schedule study and bonus sections), which were checked against the generated results. The dataset came from the Kaggle source [2]; scikit-learn was used only for comparison models, metric cross-checks and the optional bonuses.

References

[1] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter, "Pegasos: primal estimated sub-gradient solver for SVM," *Mathematical Programming*, vol. 127, no. 1, pp. 3–30, 2011.

[2] sehriyarmemmedli, "Binaaz sale project," Kaggle, dataset version 1. Dataset page (accessed Oct. 15, 2026).